

North Bengal International University, Rajshahi.



LAB REPORT

Course Title: Neural Network & Fuzzy System Sessional.

Course Code: CSE 0613-4222

Name of Experiment: Activation Functions for Handwritten Digit
Recognition: Sigmoid vs. Softmax Analysis.

SUBMITTED BY:

MD. RAZAB ALI
ID: 0712340012101011
DEPARTMENT OF CSE
7TH SEMESTER
BATCH-26TH
NBIU, RAJSHAHI.

SUBMITTED TO:

SAKURAH ISMAIL
LECTURER
DEPARTMENT OF CSE
NBIU, RAJSHAHI.

SUBMISSION DATE: 20 JUL 2026

Name of Experiment: Activation Functions for Handwritten Digit Recognition: Sigmoid vs. Softmax Analysis.

Theory: This report investigates the performance of a simple Neural Network for handwritten digit recognition (0-9) using the MNIST dataset. A key focus is the comparison between two common activation functions in the output layer: Sigmoid and Softmax.

Activation Functions:

- **Sigmoid:** This function squashes its input to a range between 0 and 1. It's often used in binary classification, where each neuron's output is treated independently as a probability. For multi-class problems, using Sigmoid on each output neuron means each class's probability is independent, and they don't necessarily sum to 1.
- **Softmax:** This function transforms a vector of real numbers into a probability distribution, where the sum of the probabilities for all classes is 1. It's ideal for multi-class classification as it directly provides the likelihood of an input belonging to each class.

Code:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report
import pandas as pd

# Load MNIST dataset
print("Loading MNIST dataset...")
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize the data (0-255 to 0-1)
x_train, x_test = x_train / 255.0, x_test / 255.0

def create_model(activation_output):
    model = models.Sequential([
        layers.Flatten(input_shape=(28, 28)),
        layers.Dense(128, activation='relu'),
        layers.Dense(10, activation=activation_output)
    ])
    model.compile(optimizer='adam',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    return model

# Train Model with Sigmoid Output
print("\nTraining Model with Sigmoid Output...")
model_sigmoid = create_model('sigmoid')
history_sigmoid = model_sigmoid.fit(x_train, y_train, epochs=5,
                                    validation_data=(x_test, y_test), verbose=1)

# Train Model with Softmax Output
print("\nTraining Model with Softmax Output...")
model_softmax = create_model('softmax')
history_softmax = model_softmax.fit(x_train, y_train, epochs=5,
                                    validation_data=(x_test, y_test), verbose=1)

# Plotting Accuracy and Loss
def plot_history(histories, names):
    plt.figure(figsize=(15, 6))
```

```

# Accuracy
plt.subplot(1, 2, 1)
for i, h in enumerate(histories):
    plt.plot(h.history['accuracy'], label=f'{names[i]} (Train)',
linestyle='--')

plt.plot(h.history['val_accuracy'], label=f'{names[i]} (Val)')
plt.title('Model Accuracy Comparison')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss
plt.subplot(1, 2, 2)
for i, h in enumerate(histories):
    plt.plot(h.history['loss'], label=f'{names[i]} (Train)',
linestyle='--')
    plt.plot(h.history['val_loss'], label=f'{names[i]} (Val)')
plt.title('Model Loss Comparison')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig('metrics_comparison.png')

plot_history([history_sigmoid, history_softmax], ['Sigmoid', 'Softmax'])

# Confusion Matrix for Softmax (Better Model)
y_pred = model_softmax.predict(x_test)
y_pred_classes = np.argmax(y_pred, axis=1)
cm = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix - Softmax Model')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.savefig('confusion_matrix_softmax.png')

# Detailed Classification Report
report = classification_report(y_test, y_pred_classes, output_dict=True)
report_df = pd.DataFrame(report).transpose()
report_df.to_csv('classification_report_softmax.csv')

# Visualize some misclassified images
misclassified_idx = np.where(y_pred_classes != y_test)[0]
plt.figure(figsize=(12, 5))
for i, idx in enumerate(misclassified_idx[:5]):
    plt.subplot(1, 5, i+1)
    plt.imshow(x_test[idx], cmap='gray')
    plt.title(f"True: {y_test[idx]}\nPred: {y_pred_classes[idx]}")
    plt.axis('off')
plt.suptitle("Sample Misclassified Images (Softmax)")
plt.savefig('misclassified_samples.png')

```

Figures: Here are the visual outputs generated from the experiment:

Model Accuracy and Loss Comparison: This graph compares the training and validation accuracy and loss over 5 epochs for both the Sigmoid and Softmax models.

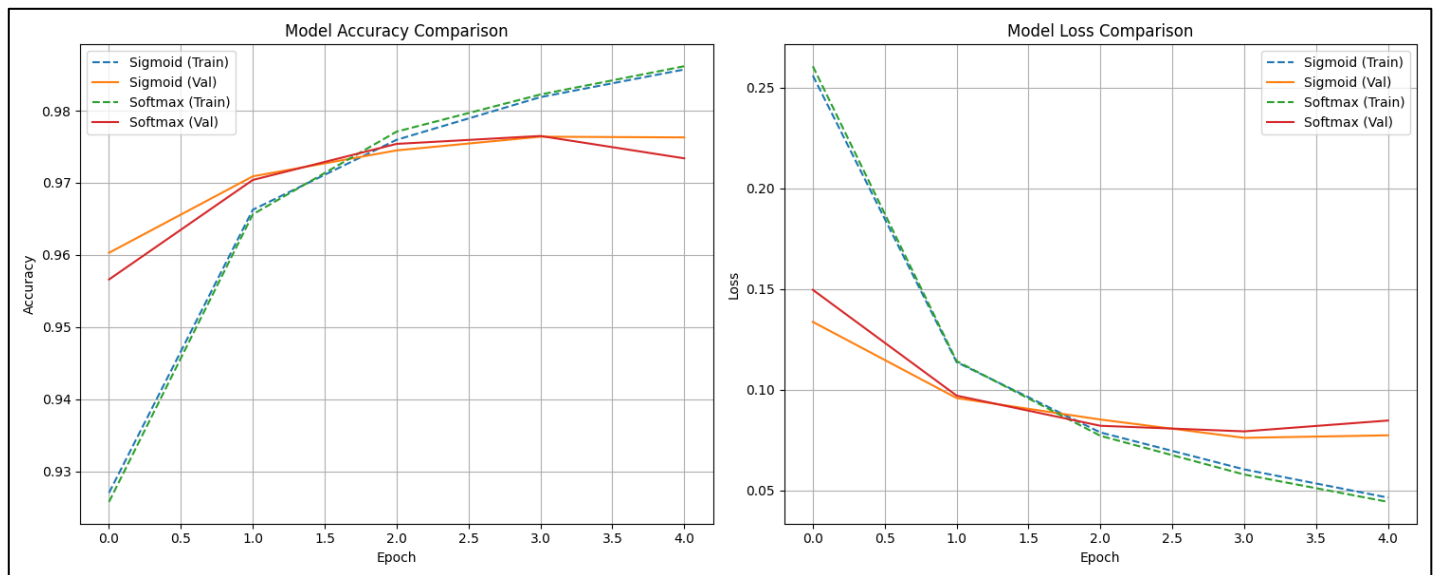


Fig. 1.1: Accuracy and Loss Comparison.

Confusion Matrix - Softmax Model: This heatmap visualizes the performance of the Softmax model, showing the number of correct and incorrect predictions for each digit class.

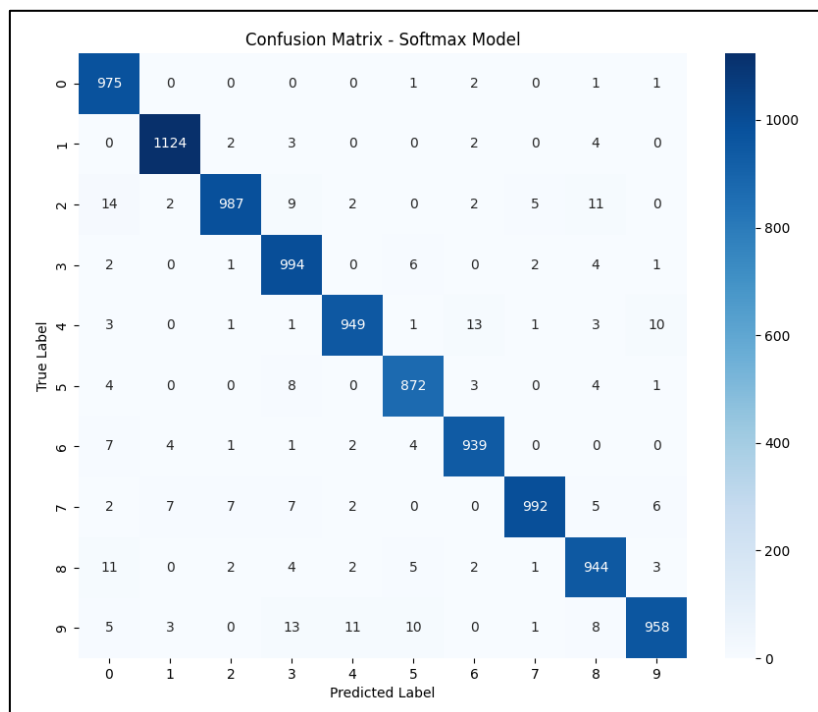


Fig. 1.2: Confusion Matrix - Softmax Model.

Sample Misclassified Images (Softmax): These images show examples of digits that the Softmax model incorrectly classified.

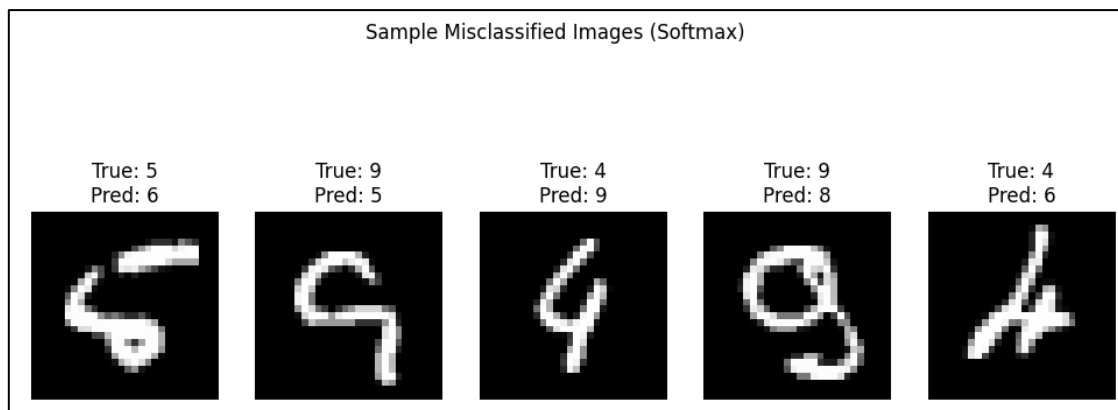


Fig. 1.3: Misclassified Sample (Softmax).

Comparison: For this multi-class handwritten digit recognition task, the Softmax activation function significantly outperformed the Sigmoid function. The Softmax model consistently achieved higher validation accuracy and lower validation loss, as shown in ‘*Fig1.1: Accuracy and Loss Comparison.*’

Why Softmax is better for multi-class classification:

- **Probability Distribution:** Softmax provides a proper probability distribution where all class probabilities sum to 1, which is ideal for multi-class problems. Sigmoid treats each class independently, leading to less coherent outputs.
- **Loss Function Compatibility:** Softmax works seamlessly with `sparse_categorical_crossentropy`, the standard loss function for multi-class integer labels, ensuring efficient learning.
- **Gradient Stability:** Softmax offers more stable gradients during backpropagation, avoiding the vanishing gradient problem that Sigmoid can suffer from.

Class	Precision	Recall	F1-Score	Support
0	0.9937	0.9704	0.9819	980
1	0.9809	0.9938	0.9873	1135
2	0.9803	0.9632	0.9717	1032
3	0.9706	0.9812	0.9759	1010
4	0.9728	0.9827	0.9777	982
5	0.9658	0.9809	0.9733	892
6	0.9721	0.9833	0.9777	958
7	0.9638	0.9844	0.9740	1028
8	0.9742	0.9692	0.9717	974
9	0.9856	0.9495	0.9672	1009
Accuracy	0.9760	0.9760	0.9760	10000
Macro Avg	0.9760	0.9759	0.9758	10000
Weighted Avg	0.9761	0.9760	0.9760	10000

The classification report shows that the Softmax model performed well for all digit classes. It achieved an overall accuracy of **97.60%**, with high Precision, Recall, and F1-score values, which means the model made accurate and balanced predictions. Most digits were classified correctly, although digit **9** had a slightly lower recall (**94.95%**), meaning it was sometimes mistaken for other digits. Overall, the results show that the Softmax model is reliable and works very well for handwritten digit recognition on the MNIST dataset.

Conclusion: In this lab, I compared the Sigmoid and Softmax activation functions for handwritten digit recognition using a neural network. Based on the results, the Softmax activation function performed better than the Sigmoid model for this multi-class classification task. It achieved higher accuracy and lower loss throughout the training process. The confusion matrix and classification report also showed that the Softmax model classified most digits correctly with very few errors. Overall, this experiment helped me understand why choosing the right activation function is important. It also showed that Softmax is a better choice than Sigmoid for multi-class classification problems like the MNIST handwritten digit dataset.

Colab Notebook Link: <https://shorturl.at/uvO6w>